

PORTFOLIO

김준우 Joon Woo Kim

DevOps / Cloud Engineer

VFX 제작 환경의 운영·자동화 경험을 Kubernetes, CI/CD, GitOps 기반 Cloud Native 환경으로 확장하고 있습니다.

EMAIL	joon@junu.dev	WEB	junu.dev
GITHUB	github.com/Vivacissimo	WIKI	wiki.junu.dev

CONTENTS

- 01

Cloud Native Delivery
4개 서비스의 배포 자동화부터 Istio 트래픽 제어, 배포 후 상태 확인까지 하나의 운영 흐름으로 연결

KT Cloud BootCamp
- 02

MoongCheap · Cloud Native Platform
Frontend · Backend · AI 서비스를 AWS EKS에 일관되게 배포하기 위한 CI/CD 및 GitOps 구조 설계

KT Cloud BootCamp
- 03

Cocoavision · VFX Pipeline / Infrastructure
DCC 환경과 Render Farm을 운영하며 Tractor 제출, PXE 설치, Ansible 노드 관리를 자동화

Professional
- 04

junu.dev Home Infrastructure
개인 서비스와 기술 위키를 직접 배포·운영하며 DNS, Container, Backup, CI/CD를 반복해서 경험하는 Home Lab

Personal

Cloud Native Delivery

4개 서비스의 배포 자동화부터 Istio 트래픽 제어, 배포 후
상태 확인까지 하나의 운영 흐름으로 연결

PERIOD	2026 · KT Cloud BootCamp
TEAM	4인
ROLE	Team Lead · Observability
SERVICES	Patient · Prescription · Appointment · Notification
PLATFORM	Kubernetes · AWS EKS

01 프로젝트 개요

CI/CD 파이프라인이 성공했다고 해서 배포가 끝난 것은 아니라고 보고, 새 버전의 실제 트래픽과 상태까지 확인하는 운영 흐름을 만들었다.

환자, 처방, 예약, 알림 4개 서비스를 대상으로 CI/CD를 구성하고, 심화 단계에서 Istio Service Mesh를 적용해 Canary와 Blue/Green 배포를 검증했다. 팀장과 Observability 담당으로서 트래픽, 오류율, 응답시간, Envoy 리소스 상태를 Prometheus/Grafana/Kiali로 확인할 수 있도록 관찰 기준을 정리했다.

PROJECT GOAL

배포 이후까지 확인하는 Delivery Flow

Build → Deploy → Traffic Shift → Observe를
하나의 절차로 연결했다.

MY ROLE

팀장 · Observability

전체 단계의 연결과 배포 후 검증 지표를 담당했다.

DEPLOY STRATEGY

Canary · Blue/Green

서비스 특성에 따라 두 가지 전환 방식을 나눠 적용했다.

KEY SIGNALS

Traffic · Error · Latency · Resource

트래픽 전환 후 서비스 상태가 악화되는지 확인할 기준을
잡았다.

02 문제 정의

PROBLEM 01

배포 성공과 서비스 정상은
다름

ArgoCD Sync가 성공해도 실제
요청이 새 버전에 정상적으로
도달하는지 별도 확인이 필요했다.

PROBLEM 02

한 번에 전환하면 장애 영향이
큼

새 버전 전체 전환은 오류가 있을 때
사용자 영향 범위가 커질 수 있다.

PROBLEM 03

운영 판단 기준이 부족

다음 Canary 단계로 진행할지
결정하려면 트래픽, 에러,
응답시간을 함께 볼 수 있어야 했다.

AS-IS → TO-BE

AS-IS

- 배포 파이프라인 성공 여부 중심 확인
- 새 버전 전체 전환 시 장애 영향 범위가 큼
- 서비스 간 호출 흐름을 개별 로그로 확인
- 배포 전후 지표 비교 기준이 분산됨



TO-BE

- GitOps 배포 후 Istio로 단계적 Traffic Shift
- Canary 단계별 상태 확인 후 다음 비율로 전환
- Kiali에서 Service Topology와 버전별 흐름 확인
- Prometheus/Grafana 지표를 공통 판단 기준으로 사용

담당 역할과 기여

TEAM LEAD

기본 → 심화 단계의 전체 흐름 정리

CI/CD, Service Mesh, Observability가 따로 보이지 않도록 실습 순서와 검증 포인트를 연결했다.

OBSERVABILITY

모니터링 Stack 구성

Prometheus, Grafana, Kiali 중심으로 서비스 상태를 확인할 환경을 구성했다.

METRIC

운영 지표 선정

트래픽, 오류율, 응답시간과 Envoy CPU/Memory를 중심으로 배포 전후를 비교했다.

SERVICE MESH

Traffic Shift 검증

예약 서비스 Canary와 알림 서비스 Blue/Green 시나리오를 팀 구성과 연결했다.

GITOPS

ArgoCD 기반 CD

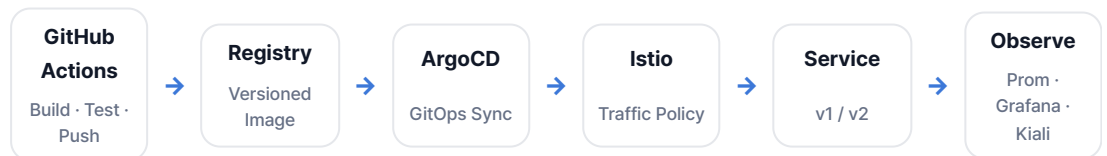
Git에 선언된 상태를 기준으로 Kubernetes 배포를 동기화하는 흐름을 사용했다.

COLLAB

SRE · Platform · Traffic 역할 협업

Observability 결과가 트래픽 전환과 장애 대응 판단에 사용될 수 있도록 역할 간 확인 기준을 맞췄다.

운영 흐름



배포 전략

Appointment

Canary 20% → 50% → 70% → 100%

새 버전의 트래픽을 단계적으로 늘리고 각 단계에서 오류율, 응답시간과 Kiali 트래픽 흐름을 확인하는 시나리오를 구성했다.

Notification

Blue / Green

두 버전을 동시에 유지한 뒤 라우팅 대상을 전환하는 방식으로 버전 교체 시나리오를 구성했다.

Security

mTLS STRICT 검토

Service Mesh 내 서비스 간 통신을 Istio 정책으로 보호하는 방식을 실습했다.

Visibility

Kiali Topology

서비스와 버전별 요청 흐름을 시각적으로 확인해 Traffic Rule 적용 상태를 검증했다.

관찰 기준

TRAFFIC

Request / Routing 비율

Canary 비율이 의도한 대로 적용되고 있는지 확인한다.

ERROR

HTTP Error Rate

새 버전 전환 후 오류 비율이 증가하는지 확인한다.

LATENCY

Response Time

버전 변경으로 응답시간이 악화되는지 비교한다.

RESOURCE

Envoy CPU / Memory

Sidecar 적용 후 Proxy 리소스 사용량을 함께 관찰한다.

결과와 배운 점

4개 서비스의 배포 자동화, 트래픽 전환, 상태 확인을 하나의 운영 시나리오로 연결했다.

성과 수치를 임의로 만들기보다, 실제로 검증한 서비스 수와 Canary 단계, 관찰 기준을 중심으로 결과를 제시했다. 이 과정에서 "배포 완료"가 아니라 "새 버전이 정상적으로 트래픽을 처리하는지"까지 확인해야 운영이 끝난다는 관점을 정리했다.

SERVICE

4개 서비스

Patient · Prescription ·
Appointment · Notification

CANARY

4단계 전환

20 → 50 → 70 → 100%

OBSERVE

4개 핵심 관찰 영역

Traffic · Error · Latency · Resource

Tech Stack: Kubernetes · AWS EKS · GitHub Actions · ArgoCD · Istio · Envoy · Prometheus · Grafana · Kiali · Jaeger

MoongCheap

Frontend · Backend · AI 서비스를 AWS EKS에
일관되게 배포하기 위한 CI/CD 및 GitOps 구조 설계

현재 진행 중인 통합 프로젝트로, 실제 구현이 완료된 부분과 설계/구성 중인 부분을
구분해 기술했다.

PERIOD	2026 · 진행 중
ROLE	CI/CD · Kubernetes · GitOps
SCOPE	Frontend · Backend · AI · Platform
PLATFORM	AWS · EKS · ECR
STATUS	GitOps 구조 및 Helm/ArgoCD 구성 진행

01 프로젝트 개요

애플리케이션 팀이 각자의 코드를 개발하더라도, 빌드 이후 배포 과정은 동일한 규칙으로
흘러가도록 만드는 것이 목표였다.

Frontend, Backend, AI가 서로 다른 저장소에서 개발되는 환경에서 CI 결과물인 Container Image를
ECR에 저장하고, GitOps 저장소의 배포 상태를 변경한 뒤 ArgoCD가 EKS에 반영하도록 구조를 설계했다.
서비스 코드와 배포 설정의 책임을 분리하고, dev/prod 환경이 늘어나도 YAML을 반복 복사하지 않는 구조에
초점을 맞췄다.

PROJECT GOAL

반복 가능한 배포 구조

서비스마다 별도의 배포 방식을 만들지 않고 공통
규칙으로 배포한다.

MY FOCUS

Build 이후부터 Cluster 반영까지

CI, Image Registry, Helm, GitOps, ArgoCD 사이의
연결을 담당했다.

COLLABORATION

서비스 팀과 배포 인터페이스 정의

애플리케이션 코드와 배포 설정의 소유 영역을 나눴다.

KEY OUTPUT

공통 Chart + 환경별 Values + ApplicationSet

서비스/환경이 늘어날 때 변경 범위를 최소화하는 구조를
만들었다.

02 문제 정의

PROBLEM 01

서비스별 YAML 중복

Deployment와 Service를
서비스마다 복제하면 공통 변경 시
수정 지점이 늘어나고 설정 차이가
생기기 쉽다.

PROBLEM 02

환경별 설정 혼재

dev/prod의 도메인, 리소스,
이미지 태그 차이를 한 파일에서
관리하면 운영 환경 변경 범위가
불명확해진다.

PROBLEM 03

배포 책임 경계 불명확

서비스 저장소가 Kubernetes
설정까지 모두 가지면 개발 코드와
운영 상태 변경이 섞이고 협업 시
소유권이 모호해진다.

03 구축 방향

신규 구축 프로젝트이므로 기존 구조를 억지로 비교하기보다, 서비스와 환경이 늘어날 때
유지보수 가능한 배포 구조를 만드는 데 초점을 맞췄다.

STANDARDIZE

공통 Workload 구조

Deployment와 Service의 공통 부분은 Helm Chart로 묶고 서비스별 차이만 Values로 전달한다.

SEPARATE ENV

dev / prod 설정 분리

도메인, 리소스, Image Tag, Secret 참조처럼 환경에 따라 달라지는 값을 명시적으로 분리한다.

SEPARATE RESPONSIBILITY

Application과 배포 상태 분리

서비스 코드는 각 팀 Repository에서, Kubernetes 배포 상태는 GitOps Repository에서 관리한다.

TRACEABILITY

Commit과 배포 버전 연결

Commit SHA 기반 Image Tag와 GitOps 변경 이력을 통해 어떤 코드가 배포되었는지 추적할 수 있게 한다.

04

담당 역할과 기여

CI

Jenkins 기반 Build/Test/Image Push 흐름

서비스 변경을 감지해 Container Image를 생성하고 ECR에 Push하는 흐름을 기준으로 설계했다.

KUBERNETES

공통 Deployment/Service 템플릿

서비스별 차이를 Values로 주입하도록 Helm Chart 구조를 정리했다.

GITOPS

배포 상태 전용 Repository 구조

charts, environments, argocd, platform 영역을 분리해 역할과 변경 범위를 명확히 했다.

ARGOCD

ApplicationSet 기반 배포 생성

서비스와 환경이 늘어날 때 Application YAML을 반복 작성하지 않도록 구성했다.

VERSIONING

Commit SHA 기반 Image Tag

CI 결과와 GitOps 상태를 같은 버전 기준으로 연결해 배포 추적성을 높였다.

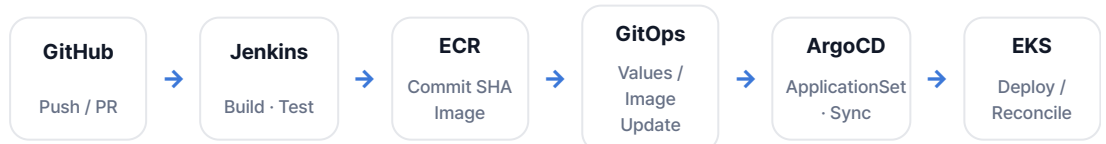
COLLAB

팀 간 전달 항목 정의

서비스 팀이 제공해야 할 포트, Health Check, 환경 변수, Image 정보와 Infra/GitOps 측 책임을 구분했다.

05

배포 구조



```

gitops/
├── charts/
│   └── moongcheap-service/
│       ├── Chart.yaml
│       ├── values.yaml
│       └── templates/
├── environments/
│   ├── dev/
│   └── prod/
├── argocd/
│   └── applicationset.yaml
└── platform/
    ├── monitoring/
    ├── logging/
    └── ingress/

```

06

설계 판단

DECISION 01

Application Repo와 GitOps Repo 분리

애플리케이션 변경과 운영 상태 변경의 리뷰 범위를 분리하고, ArgoCD가 GitOps 저장소를 단일 배포 소스로 보게 했다.

DECISION 02

서비스별 Chart 대신 공통 Chart

현재 서비스의 Workload 구조가 유사하므로 중복 Chart보다 하나의 공통 Chart와 Values 차이가 관리 비용이 낮다고 판단했다.

DECISION 03

Application 대신 ApplicationSet

서비스 × 환경 조합이 증가할 것을 고려해 개별 Application 생성보다 선언 규칙을 반복 사용하는 방식을 선택했다.

DECISION 04

dev / prod Values 분리

운영 리소스, 도메인, Secret 참조와 같은 환경 차이를 명시적으로 분리해 배포 사고 가능성을 줄이는 방향으로 구성했다.

07

산출물과 검증

GitOps 구조

charts / environments / argocd / platform 분리

배포 구성과 플랫폼 애드온의 변경 범위를 나눴다.

Helm

공통 Chart + 서비스별 Values

Deployment/Service의 공통 구조와 서비스별 설정을 분리했다.

ArgoCD

ApplicationSet 패턴

서비스와 환경이 추가되어도 동일한 생성 규칙을 재사용할 수 있게 했다.

CI/CD Contract

Image Tag와 GitOps Update 기준

CI 산출물과 실제 배포 상태를 연결할 수 있도록 버전 전달 방식을 정리했다.

운영 확장

platform 영역 분리

Monitoring, Logging, Ingress 등 애플리케이션과 다른 생명주기의 구성을 별도 관리하도록 했다.

성과를 과장하지 않고, 현재 프로젝트에서 실제로 확보한 구조와 판단을 중심으로 정리했다.

아직 진행 중인 프로젝트라 운영 지표 대신, 서비스가 늘어날 때 반복 작업을 줄일 수 있는 배포 구조를 코드로 명확히 한 점을 핵심 결과로 본다.

SCOPE**3개 애플리케이션 영역**

Frontend · Backend · AI

ENVIRONMENT**dev / prod 분리**

환경별 Values 기준

DEPLOYMENT MODE**공통 Chart +
ApplicationSet**

중복 Manifest 최소화

Tech Stack: AWS · EKS · ECR · Kubernetes · Helm · Jenkins · ArgoCD · GitOps · Terraform

Cocoavision

작업자가 사용하는 DCC 환경과 Render Farm을
운영하며 반복 작업을 자동화하고 제작 중단 원인을 해결

PERIOD	2022.09 - 2026.03
ROLE	Pipeline / IT Infrastructure
USERS	Artist · Production · External Vendor
ENV	Linux · DCC · Render Farm
FOCUS	Troubleshooting · Automation · Standardization

01 업무 개요

사용자 문의를 한 번 해결하는 데서 끝내지 않고, 같은 문제가 반복될 때는 Pipeline이나 Infrastructure 차원에서 줄이는 방식으로 접근했다.

Houdini, Nuke, Maya 등 제작 도구와 Linux 기반 Render Farm 사이에서 작업이 안정적으로 이어지도록 환경을 운영했다. Render 실패, 경로/마운트 문제, 플러그인/버전 차이 등 제작 중 발생하는 이슈를 분석했고, 반복되는 설치와 설정 업무는 PXE와 Ansible로 자동화했다. 외주사와의 작업에서는 파일, 환경, 전달 방식의 차이에서 발생하는 문제를 기술적으로 설명하고 대응했다.

OPERATE

Linux 제작 환경과 Render Farm

작업 중단을 만드는 환경 문제를 확인하고 복구했다.

AUTOMATE

PXE · Ansible · Python

반복 설치, 노드 설정, 제출 과정을 자동화 가능한 형태로 바꿨다.

TROUBLESHOOT

DCC · Plugin · Path · Mount

로그와 실행 환경 차이를 기준으로 문제 범위를 좁혔다.

COMMUNICATE

Artist / Production / Vendor

기술적 원인과 필요한 조치를 비개발 사용자도 이해할 수 있게 전달했다.

02 핵심 문제

PROBLEM 01

작업자마다 다른 제출 방식

Render Farm 제출 과정이 수동 절차에 의존하면 누락과 설정 차이가 발생하기 쉽다.

PROBLEM 02

노드별 반복 설정

수십 대 규모의 Render Node를 개별로 관리하면 동일 작업이 반복되고 설정 편차가 생긴다.

PROBLEM 03

PC 초기 설치 반복

USB 기반 설치 장비가 늘어날수록 사람이 직접 해야 하는 단계가 증가한다.

03 Case 01 · Tractor Submit

AS-IS

- 작업자가 제출 정보를 직접 구성
- DCC마다 제출 절차가 달라 실수 가능
- Job 규칙을 사용자에게 의존



TO-BE

- Houdini / Nuke 안에서 Submit
- 작업 정보를 Tractor Job 형식으로 변환
- 공통 제출 규칙을 Pipeline Tool에 반영

내 역할

Custom Render / Submit Node 구성

DCC에서 필요한 Job 정보를 수집하고 Render Farm으로 전달하는 흐름을 만들었다.

판단

사용자 교육보다 도구에 규칙을 넣는 방식

작업자마다 절차를 기억하게 하기보다 제출 도구가 동일한 입력과 Job 형식을 만들도록 했다.

결과

Submission Workflow 표준화

수동 Job 생성 단계를 줄이고 DCC 안에서 일관된 제출 경로를 제공했다.

04

Case 02 · PXE

AS-IS

- USB로 장비별 OS 설치
- 작업자가 직접 초기 세팅 반복
- 설치 절차 차이 발생 가능



TO-BE

- Network Boot 기반 설치
- PXE + DHCP/HTTP + Kickstart
- 초기 설치 절차 표준화

내 역할

PXE 부팅과 자동 설치 환경 구성

네트워크 부팅부터 설치 파일 제공, Kickstart 기반 자동 설치까지 연결했다.

판단

장비 수가 늘어날수록 반복 작업이 커지는 문제를 Infrastructure로 해결
개별 작업 속도를 높이는 대신 사람이 개입해야 하는 단계를 줄였다.

결과

USB 중심 설치 → 네트워크 기반 설치

PC 초기 세팅을 동일한 절차로 반복할 수 있는 기반을 만들었다.

05

Case 03 · Ansible

AS-IS

- Render Node마다 SSH 접속 후 설정
- 패키지/마운트/도구 배포 반복
- 노드별 설정 편차 가능



TO-BE

- Inventory로 대상 노드 그룹화
- Playbook으로 공통 작업 선언
- 동일한 설정을 여러 노드에 반복 적용

대상

수십 대 규모 Linux Render Farm

자동화

패키지 설치 · Mount · 팀별 Tool 배포 등

반복 운영 작업을 Playbook 단위로 정리했다.

결과

서버별 수동 작업을 재사용 가능한 운영 코드로 전환

동일 작업을 여러 노드에 일관되게 적용할 수 있게 했다.

06

장애 대응 사례

ISSUE

로컬에서는 정상인데
Farm에서 반복 실패

아티스트 작업이 로컬에서는
열리지만 Render Farm 제출 후
실패해 제작이 중단되는 상황.

ANALYSIS

실행 환경 차이를 기준으로
범위 축소

Tractor Log, DCC/Plugin 버전,
환경 변수, 파일 경로, Mount
상태를 순서대로 비교했다.

ACTION

설정 수정 + 반복 방지

문제 설정을 수정한 뒤 같은 유형이
다른 노드에 반복되지 않도록 운영
절차와 자동화 대상을 보완했다.

장애 대응에서 가장 중요하게 본 것은 "어디에서만 실패하는가"였다. 로컬과 Farm의 차이를
비교해 원인을 좁히고, 해결 후에는 같은 문제를 운영 코드나 절차로 줄이는 방식으로

접근했다.

07

성과와 연결점

VFX에서 경험한 운영 문제를 DevOps에서 사용하는 방식과 자연스럽게 연결할 수 있었다.

Render Farm 운영은 다수 노드의 일관된 설정, 장애 로그 분석, 반복 작업 자동화, 사용자와의 협업이 필요한 환경이었다. 이 경험은 이후 Kubernetes, CI/CD, IaC를 학습할 때 단순 도구 사용보다 운영 문제를 어떤 구조로 줄일지 생각하는 기반이 됐다.

FARM

수십 대 규모 노드

Linux Render Node 운영

AUTOMATION

3개 핵심 자동화 영역

Submit · Provisioning ·
Configuration

WORK STYLE

Fix → Standardize → Automate

반복 이슈를 구조적으로 줄이는 방식

Tech Stack: Linux · Houdini · Nuke · Maya · Tractor · Python · Ansible · PXE · Unreal Engine · Blender

junu.dev Home Infrastructure

개인 서비스와 기술 위키를 직접 배포·운영하며 DNS, Container, Backup, CI/CD를 반복해서 경험하는 Home Lab

PERIOD	Ongoing
ROLE	Design · Deploy · Operate
BASE	Raspberry Pi · Linux · NVMe
EDGE	Cloudflare Tunnel · DNS
SERVICES	Wiki · Portfolio · Web Tools

01 프로젝트 개요

일회성 실습 VM이 아니라 실제 사용하는 서비스를 계속 운영하면서 변경과 장애를 직접 경험하기 위한 개인 인프라다.

Raspberry Pi 기반 Linux 서버에서 Wiki.js와 개인 웹 서비스를 운영한다. Docker Compose로 서비스를 분리하고 Cloudflare Tunnel과 DNS로 외부 접근을 연결했다. 정적 웹 서비스는 GitHub Actions와 Self-hosted Runner를 이용해 배포하고, Wiki 데이터는 PostgreSQL과 SQL Dump 백업으로 관리한다.

02 운영 목표

GOAL 01

직접 운영하는 환경

기술을 설치 후 종료하는 실습이 아니라 지속적으로 사용하는 서비스에 적용한다.

GOAL 02

외부 공개와 보안

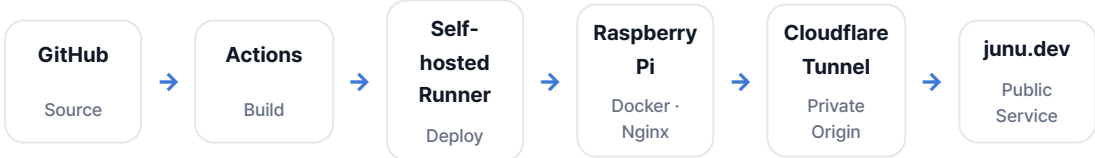
공인 포트 노출을 최소화하고 Cloudflare Tunnel을 통해 서비스별 도메인을 연결한다.

GOAL 03

반복 배포 자동화

정적 파일을 서버에 수동 복사하는 대신 Git Push를 기준으로 배포 흐름을 만든다.

03 구성



WIKI

Wiki.js + PostgreSQL

기술 학습 문서와 데이터를 분리해 운영하고 SQL Dump 백업을 유지한다.

WEB

Nginx + Static Files

개인 사이트와 웹 도구의 정적 결과물을 서비스한다.

EDGE

Cloudflare Tunnel

Origin을 직접 공개하지 않고 도메인별 서비스 경로를 연결한다.

STORAGE

microSD → NVMe

지속적인 서비스 운영을 위해 저장장치 구조도 개선했다.

담당 작업

CONTAINER

Docker Compose 서비스 구성

Wiki.js/PostgreSQL 등 서비스를 컨테이너 단위로 분리하고 볼륨을 통해 데이터를 유지한다.

NETWORK

DNS · Tunnel · Nginx

junu.dev 하위 도메인을 서비스별로 연결하고 외부 접근 경로를 관리한다.

CI/CD

GitHub Actions → Self-hosted Runner

Push 이후 빌드 결과를 홈 서버의 정적 서비스 경로에 반영하도록 배포 흐름을 구성했다.

BACKUP

PostgreSQL SQL Dump

Wiki 데이터가 컨테이너 재생성에 영향을 받지 않도록 데이터와 백업 절차를 분리했다.

AS-IS → TO-BE

AS-IS

- 서비스마다 수동 실행·배포
- 외부 접근을 위해 포트 개방 필요
- 정적 파일 변경 때 서버 직접 반영
- 백업 절차와 위치가 분산



TO-BE

- Compose 기반 서비스 정의
- Cloudflare Tunnel로 외부 연결
- GitHub Actions 기반 자동 배포
- DB Dump 중심 백업 절차 정리

Personal Dev Tools

개인 작업에서 반복되던 기능을 작은 웹 도구로 분리해 직접 배포하고 운영한다.

MASKLY

Image Privacy Tool

이미지 개인정보와 메타데이터를 정리하고 처리하기 위한 웹 도구를 제작해 maskly.junu.dev로 운영한다.

TRANSFORMER

Browser File Convert Tool

브라우저에서 파일 확장자를 변환할 수 있는 도구를 제작해 transformer.junu.dev로 운영한다.

DELIVERY

GitHub 기반 배포 흐름

코드 변경 이후 개인 서버의 정적 서비스 경로에 반영되는 배포 흐름을 구성했다.

OPERATION

서비스별 Subdomain 운영

Nginx와 Cloudflare 기반 외부 접근 구조 안에서 도구를 독립 서비스로 운영한다.

배운 점

서비스를 직접 운영하면서 배포 이후의 책임까지 경험하고 있다.

코드를 올리는 것뿐 아니라 DNS, 네트워크, 프로세스 재시작, 데이터 복구, 배포 실패처럼 실제 서비스 상태를 유지하기 위해 확인해야 하는 영역을 반복해서 경험하는 환경이다.

SERVICE

다수 Endpoint

portfolio · resume · wiki · tools

DEPLOY

Git Push 기반 자동화

정적 서비스 배포

OPERATE

Self-hosted Lab

실사용 기반 운영 경험

